

GridSathi — Antigravity IDE Build Plan (Real Data, Phase-by-Phase)

How to use this: each phase has (1) any manual step you must do yourself outside the IDE — mostly account/download steps no AI agent can do for you — and (2) a ready-to-paste **PROMPT** block for your Antigravity agent. Do phases in order; don't skip the manual steps, the agent can't fabricate real data for you (and shouldn't).

PHASE 0 — Project Setup

Manual step: Open Antigravity IDE → New Project → name it `GridSathi` → Python environment.

PROMPT:



```
Set up a new Python project called GridSathi for an electricity demand forecasting app.  
Create this folder structure:  
data/raw/  
data/processed/  
src/features/  
src/models/  
app/  
model/  
Create a requirements.txt with: pandas numpy scikit-learn xgboost lightgbm streamlit  
plotly requests holidays kaggle joblib  
Then create a .gitignore that excludes data/raw/, data/processed/, model/, and any  
kaggle/kaggle.json credentials file.
```

PHASE 1 — Get the Real Dataset (manual download, cannot be skipped)

This is the step the earlier fabricated workflow skipped. Do this yourself first:

Create a free account at kaggle.com if you don't have one.

Go to Account Settings → API → "Create New Token" → downloads `kaggle.json`.

Place `kaggle.json` in `~/.kaggle/kaggle.json` (or wherever Antigravity's terminal expects it — check with `echo $HOME`).

Go to the dataset page: "**Delhi hourly power consumption (2000-2023)**" by manilmodi on Kaggle, and note the exact dataset slug shown in the URL.

PROMPT (after you've placed kaggle.json):



In the terminal, run these commands to download the real Delhi power dataset from Kaggle:

```
kaggle datasets download -d manilmodi/delhi-hourly-power-consumption-2000-2023
unzip delhi-hourly-power-consumption-2000-2023.zip -d data/raw/
```

Then write a script `src/data/inspect_data.py` that loads whatever CSV file appears in `data/raw/`, prints `df.columns`, `df.dtypes`, `df.shape`, `df.head(10)`, and `df.isnull().sum()`. Do not assume column names — print them so I can see the real schema before we write any feature engineering code.

Run it, look at the real output, and confirm the actual column names before Phase 3 — this matters, don't let the agent guess.

PROMPT (fetch supplementary weather data — legitimate, no key needed):

 

Write `src/data/download_weather.py` that calls the Open-Meteo Archive API (<https://archive-api.open-meteo.com/v1/archive>) for latitude 28.61, longitude 77.20, requesting hourly temperature_2m, relative_humidity_2m, and windspeed_10m for the full date range covered by our power dataset (use the min/max dates found in Phase 1's `inspect_data.py` output — don't hardcode a guessed range). Save the result to `data/raw/delhi_weather.csv` with a proper datetime column.

PHASE 2 — Clean & Merge Real Data

PROMPT:

 

Write `src/data/merge_data.py` that:

1. Loads the real power consumption CSV (from `data/raw/`, using the actual column names we confirmed in Phase 1 — do not invent column names)
2. Loads `data/raw/delhi_weather.csv`
3. Merges them on their shared datetime column (resolve timezone/format mismatches if any)
4. Reports how many rows had missing values after merge, and drops or interpolates them using a documented method (print which one you used and why)
5. Saves the clean merged result to `data/processed/merged.csv`

Print the final row count. It should be in the thousands, not dozens — if it's not, stop and tell me something went wrong in the merge instead of proceeding.

PHASE 3 — Feature Engineering (time-series features, not monthly averages)

PROMPT:



Write `src/features/build_features.py` that loads `data/processed/merged.csv` and adds:

- `hour`, `dayofweek`, `month` extracted from the `datetime` column
- `is_weekend`, and `is_holiday` using the Python ``holidays`` package for India (`holidays.India(years=range(start_year, end_year+1))` based on our actual data's year range)
- cyclical encodings: `hour_sin/hour_cos`, `month_sin/month_cos`
- lag features on the `load` column: `lag_1h`, `lag_24h`, `lag_168h` (prior week same hour)
- a 24-hour rolling mean of `load` (shifted by 1 to avoid leakage)
- `heat_index` = `temperature` + `0.5 * humidity`
- `ac_load_proxy`: exponential term that activates above `30°C`

Drop rows with `NaN` from the lag/rolling operations. Save to `data/processed/features.csv`.
Print the final feature list and row count.

PHASE 4 — Train the Model (proper time-based validation)

PROMPT:

 

Write `src/models/train_model.py` that:

1. Loads `data/processed/features.csv`
2. Splits data by `TIME`, not randomly — use the last 3 months of the dataset as the test set, everything before that as training. Print the actual date ranges of train and test sets.
3. Trains an `XGBRegressor` to predict the `load` column using all engineered features (`n_estimators=300`, `max_depth=5`, `learning_rate=0.05`)
4. Evaluates on the test set: print `MAE`, `RMSE`, and `MAPE`
5. Prints the top 10 features by importance
6. Saves the trained model to `model/xgb_model.pkl` using `joblib`

Do not hardcode or assume the resulting metrics anywhere else in the project — whatever this script prints is the real number we report to judges.

Run this and read the actual printed metrics before moving on — these are the numbers that go in your presentation, not placeholders.

PHASE 5 — Build the App (Streamlit, single file — frontend + backend combined)

PROMPT:



Create `app/main.py`, a Streamlit app for GridSathi that:

1. Loads `model/xgb_model.pkl` and `data/processed/features.csv` on startup
 2. Shows a sidebar where the user inputs: temperature, humidity, hour of day, day of week, and whether it's a holiday
 3. On clicking "Predict Demand", constructs a feature row matching the exact columns the model was trained on (reuse the feature engineering logic from `src/features/build_features.py` rather than duplicating it — refactor that file so its functions can be imported and reused here for a single live input row)
 4. Displays the predicted MW value, and a "duck curve risk" flag if the predicted hour is between 18:00-21:00 in summer months with high AC load proxy
 5. Below that, plots the real historical load curve (from `features.csv`) using Plotly, so judges see actual Delhi data, not synthetic data
 6. Add a "What-If" section: let the user shift temperature by +/- a few degrees and show the resulting change in predicted demand as a delta, computed live from the model — not a scripted/precomputed number
- Keep the whole app in this one file for simplicity.

PROMPT (local test):



Run: `streamlit run app/main.py`

Confirm the app loads without errors and that a prediction actually changes when change the sidebar inputs. If the predicted value doesn't change, debug the feature construction — it's likely not matching the training feature order/columns.

PHASE 6 — Push to GitHub & Deploy

Manual step: Create a GitHub repo named `gridsathi`.

PROMPT:



Initialize git in this project, create a `README.md` describing GridSathi, the real data source used (Delhi hourly power consumption 2000-2023, Kaggle), and how to run it locally. Commit everything except what's in `.gitignore`. Push to the GitHub repo I've created at [paste your repo URL here].

Manual step: Go to `share.streamlit.io` → connect your GitHub repo → deploy `app/main.py`. This gives you a live public URL for the demo — no local server juggling on presentation day.

PHASE 7 — Prepare an Honest Demo Script

PROMPT:



Write a demo script in docs/demo_script.md for presenting GridSathi to judges. Pull the ACTUAL metrics from our Phase 4 training output (MAE, RMSE, MAPE — ask me for these numbers if you don't already have them in context, don't invent them) and the actual date range of our dataset. Structure it as:

1. Hook: state the real problem using verified figures (Delhi's 2026 peak demand context — I'll provide the specific number and source)
2. What we built: one sentence
3. Live demo steps: exact clicks to make in the deployed app
4. Our real model performance: state the actual MAPE/MAE, not a target number
5. Known limitations: data recency, weather forecast uncertainty compounding into demand uncertainty, no live SCADA/SLDC integration yet
6. Future scope: how this would plug into real-time grid operations

Do not include any cost-savings rupee figures unless I explicitly provide a verified tariff source and the calculation — leave a placeholder marked [NEEDS VERIFIED SOURCE] if that section is wanted.

Rules for using these prompts

Run phases **in order** — each depends on real output from the previous one (real column names, real date ranges, real metrics).

Whenever a prompt says "don't invent/assume/hardcode" — that's not filler, it's the exact mistake the original fabricated workflow made. Read the agent's actual output before proceeding to the next phase.

If Antigravity's agent ever fills in a plausible-looking number instead of running code to get a real one, stop and ask it to actually execute the step.